Development Plan
Software Engineering

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

Table 1: Revision History

Date	**Developer(s)**	**Change**
28 September 2026	Shivya Mehta	Workflow Plan, Project Decomposition and Scheduling, Expected Technology, Coding Standard
28 September 2026	Anindita Bornomala	IP to protect, Copyright License, Team Meeting Plan, Team Communication Plan
28 September 2026	Harmanpreet Singh Sagar	POC demonstration plan, Reflection and Team Charter Appendices
28 September 2026	Vanessa Lai	Confidential Information, Team Member Roles, Workflow plan - use of CI/CD, Use of AI/LLMs
10 November 2026	Harmanpreet Singh Sagar	Narrowed the POC demonstration plan to the single-outline extraction pipeline (peer review #36, TA feedback); justified the choices and limitations in Expected Technology (TA feedback)
...	...	...

This document describes the team's development plan for the CAS Midterm and Assignment Optimization system. The system is designed to collect assessment information from CAS course outlines, identify periods where students may have a high assessment workload, suggest alternative assessment dates to professors, and check course outlines against McMaster's course outline policy and CEAB accreditation requirements.

The document first covers topics related to confidential information, intellectual property, and the copyright license for the project. It then explains how the team plans to work together, including meeting expectations, communication methods, team member roles, and the Git and GitHub workflow that will be used throughout development. The document also outlines the project schedule and task breakdown, the proof of concept plan, the technologies expected to be used, the team's approach to AI and LLMs, and the coding standards that will be followed. The appendices include the team's generative AI use disclosure, a reflection on the deliverable, and the team charter.

1 Confidential Information?

Yes, the project may involve confidential information related to sensitive CAS department data that will be determined later during future supervisor meetings. This information will be kept confidential and will not be shared publicly or uploaded to third-party websites without authorization.

2 IP to Protect

Intellectual Property (IP) transfer for this project is marked as "Maybe", and is still subject to further discussion with the CAS administrative supervisors (Matt Vonk, Connie Carrabs). The team will ensure that any IP-related concerns are addressed appropriately, and in the event that there is IP to protect (ex. due to potential internal administrative workflows), all team members will sign the "Intellectual Property Guide Acknowledgement" and abide by any formal IP agreements established with the CAS supervisors.

3 Copyright License

The team is adopting the MIT License for the open-source base components of the codebase, located in the root directory of our repository in the LICENSE file. The MIT License provides high flexibility for software reuse while granting the CAS Administration team full rights to modify, deploy, and maintain the software indefinitely without restriction.

4 Team Meeting Plan

4.1 Team Internal Meetings

- **Frequency & Time:** The team will meet weekly on Mondays, Wednesdays, and Thursdays at 5:30 PM (during lecture hours), as well as Wednesdays from 2:30 PM to 4:30 PM (tutorial hours). All other meetings will be scheduled as needed, with a minimum 24-hour notice to all team members.

- **Location:** All in-person meetings will take place either in lecture or at a nearby campus library study room. Virtual meetings will be conducted via Microsoft Teams.

- **Structure:** Each meeting will have a designated Meeting Chair (will rotate weekly) to enforce the agenda and a Notetaker to keep track of meeting minutes and action items. Meetings will follow a structured agenda consisting of status updates, GitHub Kanban board review, roadblock discussions, and assigning action items post-meeting.

4.2 Supervisor/TA Advisor Meetings

- **Frequency & Time:** The team will meet bi-weekly for about 30 to 60 minutes with CAS supervisors (Matt Vonk, Connie Carrabs), with each meeting being scheduled within 24 hours of the previous meeting. Additional meetings may be scheduled as needed, but this is the current plan to ensure consistent communication and feedback with the supervisors, while considering their availability. The team will also engage in 20-minute meetings with the TA advisor (Lucas Dutton) on Wednesdays, ranging from 2:30 PM to 4:30 PM (tutorial hours) to discuss project progress, roadblocks, and ask questions related to the project. All TA meetings have been scheduled in advance, as per the Software Engineering Capstone calendar (typically bi-weekly, with exceptions for events such as Proof-of-Concept demos).

- **Location:** In-person meetings with the CAS supervisors will take place at a location convenient to the supervisors (likely a meeting room booked in advance). Meetings with the TA advisor will be held in-person at ITB 120. If a team member or advisor cannot attend in-person, a hybrid meeting option can be arranged (case-by-case basis). All virtual meetings will be conducted via Microsoft Teams.

- **Structure:** The team will distribute a meeting agenda prior to each supervisor meeting. The Chair will lead the presentation and discuss recent project milestones and progress, while the Notetaker documents meeting minutes and feedback from the supervisors/advisor.

5 Team Communication Plan

- **Asynchronous Communication:** The team will use Discord as their primary day-to-day communication channel for quick questions, adjustments and review requests, and general project discussions. Asynchronous communication with supervisors and the TA advisor will be conducted via email, with a response time of 24 hours during weekdays.
- **GitHub Issues & Kanban Project Board:** All project tasks, bugs, and feature additions will be tracked using GitHub Issues. Team members will create issues on the CAS Course Optimization Project Kanban Board for unassigned tasks prior to working on them. Issues will be tagged with labels (ex. `documentation`), and linked to the relevant pull requests.
- **Code Reviews & Pull Requests:** Formal communication regarding code changes will occur within GitHub Pull Requests. As each pull requires passing CI checks and at least one review approval from another team member, this will provide the opportunity to provide feedback and ensure code quality before merging the changes into the main branch.
- **Documentation & File Sharing:** Draft deliverables and meeting notes will be maintained using Google Docs, while official source code and documentation will be kept within the primary GitHub repository.

6 Team Member Roles

The team will rotate roles at each milestone to ensure that all members remain engaged across different aspects of the development process and gain a breadth of experience. All team members will contribute to documentation and meeting notes, actively participate in GitHub reviews, and decision-making. Role assignments will be revisited at each milestone to accommodate changing priorities, and ensure that all team members gain experience across different areas of the project.

The roles we anticipate during our project include:

- **Communication Lead**: Leads communication with stakeholders, prepares meeting agendas and notes for TA and supervisor meetings, records decisions during meetings, and creates action items for team members. This role requires strong communication, organization, and collaboration skills to ensure that stakeholder feedback is accurately communicated to the team.
- **Research & Requirements Lead**: Reviews relevant peer-reviewed research, analyzes project requirements, and establishes the team's weekly priorities. This role requires requires skills in research, requirements analysis, and technical writing, as well as an understanding of the academic policies and accreditation requirements relevant to the CAS department.

- **Technical Lead**: Supports architectural and implementation decisions and helps the team debug significant technical issues and communicates with TAs when additional assistance is needed. This role requires software engineering expertise in areas such as system design, database management, software architecture, and development workflows. The Technical Lead also helps ensure that technical decisions align with project requirements and maintainability goals.

- **UI/UX & Product Lead**: Tests weekly product iterations, collects feedback, and focuses on human-computer interaction (HCI) and user experience. This role requires knowledge of user centered design, interface design, usability testing and understanding the needs of the users. This role is related to the course SFWRENG 4HC3, which all team members are taking currently.

The team brings a diverse range of technical and domain expertise to the project, including software architecture, databases, full-stack development, machine learning, software testing, and human-computer interaction. This overlap allows team members to support one another and take on responsibilities beyond their assigned roles when needed.

Summary of the team's existing domain expertise:

Team Members	**Domain Expertise**
Shivya Mehta	Databases, System Design, APIs, Fullstack, Software Requirements, Git, Documentation, HCI
Harmanpreet Singh Sagar	Software Testing, Backend, Git, Databases, Software Requirements, Hardware, Documentation, HCI
Vanessa Lai	Software Architecture, Machine Learning/NLP, Fullstack, DevOps, UI/UX, Software Requirements, API, Documentation, HCI
Anindita Bornomala	Machine Learning, AI, Git, APIs, Fullstack, Software Requirements, Documentation, Software Testing, HCI

Table 2: Team Members and Domain Expertise

7 Workflow Plan

The team will use GitHub for version control, issue tracking, project planning, and code review. The `main` branch will contain stable, production-ready code, while new features and fixes will be developed on separate branches. Each branch will address a specific GitHub Issue and will be merged into `main` through a pull request (PR). The PR will describe the changes, identify the linked issue, and summarize the testing that was performed. Before the PR is merged, at least one team member other than the author must review and approve it.

Branch Type	Format	Example
Feature	`feature/<description>`	`feature/course-upload`
Bug Fix	`fix/<description>`	`fix/calendar-filter`
Refactor	`refactor/<description>`	`refactor/api-structure`
Documentation	`docs/<description>`	`docs/setup-guide`

Table 3: Git Branch Naming Convention

GitHub Issues will be used to track features, bugs, questions, and technical tasks. Issue templates will be created for common tasks such as feature requests and bug reports. Issues will be assigned to team members and linked to relevant milestones.

GitHub Labels such as `feature`, `bug`, `documentation`, `frontend`, `backend`, and `AI/ML` will be used to categorize and organize issues. Priority labels such as `high`, `medium`, and `low` will be used to indicate the urgency of each issue.

GitHub Projects will be used as a project board to track issues through `Backlog`, `Ready`, `In Progress`, `In Review`, and `Done` statuses. The board will be reviewed during team meetings to identify blocked or overdue work, manage dependencies, and adjust priorities.

Milestones will be used to group issues according to major project phases and deliverables, allowing the team to track progress and identify outstanding work. For example, for this project milestones will likely include `MVP`, covering course-outline data extraction and policy checking, and `Final Release`, covering assessment conflict identification, scheduling suggestions, database exports, testing, and documentation.

Continuous Integration/Deployment: GitHub Actions will be used for CI to automatically run tests (unit, integration, acceptance and regression), linting, and formatting checks on pull requests. Changes merged into `main` will be deployed automatically where practical. We will also monitor deployment results, address any failures, and use stakeholder feedback and testing results to guide future development iterations.

Development Workflow: `Issue` → `Feature Branch` → `Implementation` → `Pull Request` → `CI Checks` → `Code Review` → `Merge` → `Deployment`.

8 Project Decomposition and Scheduling

The team will use **GitHub Projects** as the primary tool for scheduling and tracking project progress. A Kanban board has been created and will be used to organize tasks into *Backlog*, *Ready*, *In Progress*, *In Review*, and *Done*. Tasks will be created as GitHub Issues and assigned to individual team members. The

board will be reviewed regularly during team meetings to identify blocked tasks, adjust priorities, and ensure progress toward upcoming deliverables.

Large features will be decomposed into smaller, specific tasks that can be completed and validated within a few hours.

Work will be divided based on team members' skills, interests, and current workload. Team members will aim to spend approximately 10 hours per week on the project, including scheduled class time. Tasks will be distributed to balance the workload and avoid assigning too many dependent tasks to one team member. Team members will also collaborate on tasks that require multiple areas of expertise, such as integrating the frontend, backend, database, and LLM components.

The project schedule will be organized around the major course deliverables and project milestones. Only the first 9 weeks of the project are shown in detail, as the remaining weeks will be planned based on the progress made during the first half of the project. The schedule is as follows:

- **Week 4:** Problem Statement, POC Plan, and Development Plan.
  - Problem Statement, POC Plan, and Development Plan
  - Research text extraction and LLM approaches
  - GitHub repository and GitHub Projects setup
- **Week 5:**
  - Define initial system requirements and supported input formats
  - Confirm that the initial implementation supports PDF course outlines only
  - Document the conversion workflow for Word documents that must be processed
  - Collect sample PDF course outlines for development and testing
  - Evaluate PDF text extraction libraries
  - Define CEAB requirements to be checked by the system
- **Week 6:** Requirements Document Revision 0 and Hazard Analysis.
  - Finalize requirements for PDF course-outline processing and CEAB verification
  - Identify risks related to incorrect text extraction and LLM-generated results
  - Implement initial PDF text extraction prototype
  - Document direct Word document extraction as future work, including evaluation of DOCX extraction libraries or PDF conversion tools

- **Week 7:**
  - Implement initial LLM-based course information extraction
  - Define structured format for extracted course information
  - Design CEAB verification logic
- **Week 8:** V&V Plan Revision 0.
  - Develop tests for text extraction and CEAB verification
  - Define metrics for evaluating extraction accuracy
  - Create initial backend API for course outline processing
- **Week 9:**
  - Integrate text extraction with the LLM pipeline
  - Implement CEAB requirement verification
  - Store extracted course information in database
- **Week 10:** Design Document Revision -1 and CD-Fall Revision 0.
- **Weeks 11–12:** Proof of Concept demonstration and interviews.
- **Week 14:** MVP should be completed and ready for demonstration.
  - Demo MVP results and address remaining issues
  - Plan next-stage features based on stakeholder feedback
- **Week 16:** Design Document Revision 0.
- **Weeks 18–19:** Revision 0 demonstration and interviews.
- **Week 22:** V&V Report Revision 0 and CD-Winter Revision 0.
- **Week 24:** Final Demonstration (Revision 1).
- **Week 26:** EXPO Demonstration.
- **Final Submission:** Final documentation, including the source code and revised project documents.

The GitHub Project is available at: GitHub Project Link.

9 Proof of Concept Demonstration Plan

The main risk for this project is extracting reliable information from course outlines. CAS course outlines do not follow one standard format. Different instructors use different layouts, and some information may be missing or incomplete, such as "Midterm Date: TBD". Every other feature depends on this information, including the shared calendar, conflict detection, scheduling suggestions, and compliance checking. If assessment dates or weights are extracted incorrectly, the rest of the system will also produce unreliable results. For this reason, the proof of concept (PoC) is limited to the extraction pipeline for a single course outline.

The PoC is designed to test the following risks:

1. **Text extraction.** The PDF text extraction library may return text that is out of order or incomplete, especially for tables and multi-column layouts, so that a date or weight is attached to the wrong assessment before the model sees it.

2. **Structured extraction by a local model.** The language model must run on CAS servers, so it has to be a small open-weight model. Such a model may not organize the text into correct structured data, may invent a value that is not in the outline, or may follow an instruction written inside the outline text (prompt injection).

3. **Traceability.** Every extracted value is confirmed by a person before it is used. This review is only practical if each value comes with the passage of the outline it was taken from, so that the reviewer does not have to search the original document.

The PoC demonstration will show one course outline going through the whole pipeline:

1. A PDF course outline is uploaded through the web application.

2. The backend extracts the text from the PDF with PyMuPDF.

3. A locally hosted open-weight model, run through Ollama, receives the extracted text and returns JSON in a fixed schema: the name, type, date (or TBD), and weight of each assessment, the source passage for each assessment, the learning outcomes, and a list of missing fields.

4. The backend checks that the JSON matches the schema and that each source passage appears in the extracted text.

5. The frontend displays the assessments in a table, with the source passage shown beside each value and missing or TBD values clearly flagged.

Before the demonstration, the pipeline will also be run on a small set of real course outlines and scored against the team's answer key, so that the extraction accuracy can be reported alongside the live example. One outline will also be run with an instruction inserted into its text, and the output will be compared with the normal run of the same outline to show how the pipeline behaves under prompt injection.

The following features are part of the project but are **not** part of the PoC demonstration:

- Uploading a batch of outlines, persistent database storage, and the review and confirmation workflow.
- The shared calendar and workload or conflict detection. These only display and count data that the pipeline produces, so they carry little technical risk of their own.
- Scheduling suggestions with the CP-SAT solver. After the October meeting with the department, date suggestions became an extra feature rather than a core one. Translating policies into solver constraints is still a technical risk, and it will be evaluated with a reduced model after the PoC.
- McMaster policy and CEAB graduate-attribute checking. These checks are written as deterministic code that runs on the extracted data, so they depend on the pipeline being reliable first.
- LLM-generated explanations for date suggestions, login and user roles, and export.

If the PoC does not address the risks as expected, the team will review which parts were unsuccessful, determine the reasons, and discuss next steps with our supervisors. The response depends on which risk was not met:

- If text extraction fails on some layouts, the team will compare a second extraction library on those outlines, and outlines that still produce little or unusable text will be flagged for manual entry.
- If the model's accuracy is too low, the team will compare other small open-weight models and prompting strategies, which is already planned as a series of model-selection spikes. If no model is accurate enough, the system will move to a semi-automated approach where extracted values are pre-filled and then confirmed or corrected by a user. The small models tested so far do not reliably ignore instructions placed in outline text, so the system cannot rely on the model alone: the backend validates every model output before it is shown, and a person confirms every value.
- If source passages are unreliable, values without a matching passage will be treated as missing and entered by the reviewer.

In each case, the project plan and schedule would be adjusted, for example by supporting fewer outline layouts or moving features such as compliance checking to the stretch goals, so that the team can build a smaller, reliable system rather than many features that may not work well.

10 Expected Technology

This section lists the technologies we expect to use and the reasons for each choice. Three constraints drive these choices: the language model must run on CAS servers with no outline data sent to an external service, the tools must be free, and the system will be handed over to CAS, so it should be easy to deploy and maintain. These choices may change as development progresses.

10.1 Course Outline Processing

This is the highest-risk part of the system (Section 9), so these tools were compared in model-selection spikes before being chosen.

- **PyMuPDF (1.27.2.2)** extracts the text from each PDF outline. In the first spike, pdfplumber mixed up the columns of schedule tables and lost the spaces between words, while PyMuPDF kept table cells in order. It cannot read scanned pages, so outlines with little extracted text are flagged for manual entry. The version is pinned so every model is tested on the same text.

- **Ollama** runs the open-weight model locally and exposes it to the backend through an HTTP API. It meets the requirement that the model runs on CAS servers at no cost, lets the team switch models by changing one name, and can restrict the model's output to a JSON schema. On a CPU it took several minutes per outline in the spikes, so it sits behind a replaceable interface.

- **Small open-weight models** (Gemma 4 E2B and Llama 3.2 3B so far, with Qwen next) turn the extracted text into structured data, because models of this size can run on CAS hardware. Each is scored against the team's answer key and a prompt-injection test. They are less accurate than large hosted models, and none tested so far reliably ignores instructions in outline text, so the backend validates every output and a person confirms every value (AIR6 and AIR7 in the SRS [1]).

- **vLLM and CUDA** would replace Ollama if CAS provides an NVIDIA GPU, since vLLM serves many requests at once more efficiently. They will only be used once the server hardware is known.

- **Hugging Face Transformers and PyTorch** would only be used if the chosen model needs fine-tuning on CAS outlines. The spikes so far use the models without fine-tuning.

- **Google OR-Tools (CP-SAT)** will generate alternative assessment dates, now an extra feature. It is free, has a Python API, and fits the problem: dates are whole numbers of days, restricted periods are excluded values, and it can return several feasible solutions. A simple scoring method was considered but would not guarantee that every policy is respected. Each policy still has to be written as a constraint by hand.

10.2 Application

- **Python and FastAPI** provide the backend REST API. The extraction, model, and solver libraries are all Python, so the backend uses one language, and FastAPI's typed models can also check the model's JSON against the expected schema. Python is slower than compiled languages, but the time per outline is dominated by the model.

- **React, TypeScript, Vite, and Tailwind CSS** provide the frontend. The team already knows React, TypeScript catches mismatches with the backend's data at compile time, Vite gives a fast development server, and Tailwind makes it easy to apply the McMaster colour scheme.

- **MongoDB** will store courses, outlines, assessments, learning outcomes, and their source passages. The model's output is already JSON and its structure is still changing, so a document database avoids a migration for each new field. Links such as cross-listed courses and the audit trail must be kept by the application; PostgreSQL with JSON columns is the alternative if that proves hard, and the choice will be confirmed when the database is connected.

- **Docker** packages the backend, frontend, database, and model server, giving every developer the same environment and making it simple to hand the system to CAS IT.

10.3 Code Quality, Testing, and Project Management

- **Ruff** (Python) and **ESLint with Prettier** (TypeScript) lint and format the code in CI, so style is not debated in code review (Section 12).

- **Pytest** tests the backend and **Vitest with React Testing Library** tests the frontend; Vitest was chosen over Jest because it shares Vite's configuration. Extraction accuracy is measured separately against the answer key.

- **GitHub Actions** runs the linters and tests on every pull request and builds the documentation PDFs, at no cost and next to the code.

- **Git, GitHub, and GitHub Projects** are used for version control, issues, and the Kanban board (Section 7).

- **Visual Studio Code** is the shared editor; its Ruff, ESLint, and Prettier extensions apply the same rules as CI while code is written.

11 Use of AI and LLMs

The team plans to use Claude Code (version 2.1.283) and Cursor (Grok 4.5 Fast), as AI-assisted development tools. These tools will support planning, development, documentation, and collaboration throughout the project. Specifically, we will use AI to review project documentation against the checklists in the capstone project GitHub repository (https://github.com/smiths/capTemplate/tree/main/pdfs/Checklists), suggest codebase folder structures and configuration setups, and assist with debugging issues such as missing dependencies and LaTeX compilation errors. AI will also be used to organize and refine meeting minutes, summarize discussions, and extract key decisions and action items. All final decisions and implementations will remain under the team's control.

The team will independently verify all AI-generated suggestions for accuracy, completeness, consistency, and appropriateness before incorporating them into the project. AI-generated code will undergo human code review before being committed or pushed to the repository. Code will be validated through CI/CD pipelines using automated builds, linting, and unit, integration, and regression testing. Dependencies, configuration changes, and potential security issues will be reviewed carefully.

12 Coding Standard

The project will follow the following established coding standards to maintain consistency, readability, and maintainability across the codebase:

- **Frontend:** TypeScript and React code will follow ESLint and Prettier formatting rules.`camelCase` will be used for variables and functions, while `PascalCase` will be used for React components and types.
- **Backend:** Python code will follow PEP 8. `snake_case` will be used for variables and functions, and `PascalCase` will be used for classes.
- **Database:** MongoDB documents will use consistent field naming and structure. Indexes will be used for frequently queried fields, and database operations will be separated from application logic.
- **General:** Code will use descriptive names and consistent formatting. Comments will be used where necessary to explain non-obvious logic. Unused code and debugging statements will be removed before merging.
- **Testing:** Backend code will be tested using Pytest and frontend code using Vitest or Jest testing library. Core functionality, API endpoints,

and important application components will have corresponding tests where appropriate.

References

[1] S. Mehta, A. Bornomala, H. S. Sagar, and V. Lai, "Software Requirements Specification: CAS Course Deadline Management System," Team 19 (BVSH), Dept. of Computing and Software, McMaster University, Hamilton, ON, Canada, Oct. 2026, Accessed: Oct. 19, 2026. [Online]. Available: https://github.com/shivyam/cas-course-optimization/blob/main/pdfs/SRS-IEEE/SRS.pdf

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

The team used the following AI tools and how they were used:

- Claude Code (Opus 5) for reviewing and refining the document
- GPT-5.6 Luna
- GitHub Copilot - v1.0.88
- Cursor (Grok 4.5 Fast) for formatting and debugging LaTeX errors

13.2 How and Where Used

- Claude Code (Opus 5) was used to review and refine the document, suggesting improvements to structure and clarity, and assisting with grammar and style. It was used to primarily refine the Proof of Concept Demonstration Plan, and the Appendix - Team Charter sections.
- GPT-5.6 Luna was used to ensure the Expected Technology section was clear and concise, and to help identify any missing technologies that may be relevant to the project. The section was then reviewed and modified by the team to ensure accuracy and completeness.
- GitHub Copilot 1.0.89-7 was used to create a table and fix latex formatting issues in the Workflow Plan section.

13.3 Verification of AI-Generated Content

- Any output by Claude Code (Opus 5) was reviewed thoroughly and modified before its suggestions were added to the document.
- Course materials and deliverable requirements were manually reviewed to ensure that all technical assertions, requirements, and references were accurate and complete.

Appendix — Reflection

Why is it important to create a development plan prior to starting the project?

Our system has several components that depend on each other, including data extraction, the database, conflict detection, scheduling suggestions, and compliance checking. Creating a development plan helps make sure everyone on the team has the same understanding of how these components will work together and what tools and formats will be used. It also helps us divide responsibilities and agree on how we will use Git, track issues, and communicate throughout the project. Having these decisions made early reduces confusion later and allows us to focus the proof of concept on the areas that have the most technical risk.

In your opinion, what are the advantages and disadvantages of using CI/CD?

One of the main advantages of CI/CD is that it can catch problems early by automatically running tests whenever changes are made. This is useful for our project because many parts of the system are connected. For example, a change to the format of the extracted course data could affect the calendar, conflict detection, or scheduling solver. CI/CD can help identify these issues before they are merged into the main branch. It can also help keep the main branch stable and make it easier to maintain a working version of the system throughout development.

The main disadvantage is the additional time required to set up and maintain the CI/CD pipeline. It is also only as useful as the tests included in it. If the test coverage is limited, some problems may still go unnoticed. Certain parts of our project, such as evaluating the quality and accuracy of LLM-generated explanations, may also require manual testing rather than relying only on automated tests.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our group did not have any major disagreements while completing this deliverable. Most decisions were made through discussion and agreement between the team members. One topic that required more discussion was how the LLM should be used in the system. We considered using an external API, hosting an existing model on CAS servers, or potentially using our own locally hosted solution.

Since we have not yet had an official meeting with our supervisors, we do not currently know the exact privacy and hosting requirements for the project. For now, we agreed to use a hosted API for the proof of concept and only

send publicly available course outline information to it. We will also design the LLM component so that it can be replaced with a locally hosted model later if required. We plan to discuss the final hosting approach with our supervisors during our first project meeting.

Appendix — Team Charter

External Goals

Our team has four main goals for this project. First, we want to achieve a strong grade that reflects the amount of effort we put into the project throughout the year. Second, we want to create a complete software system that we can confidently discuss in future job interviews, applications, or graduate school opportunities. Third, we want to build something that is genuinely useful for the CAS department and could continue to be used after the course is finished. Ideally, the system would reduce some of the manual work currently done by administrators and professors while also helping students avoid weeks with too many assessments. Finally, we want to use the project as an opportunity to gain hands-on experience with technologies that are new to most of the team, including LLM integration, constraint optimization using OR-Tools, and full-stack web development.

Attendance

Expectations

All team members are expected to attend the weekly meetings that occur during lecture and tutorial times, and any additional meetings scheduled. Everyone should also attend meetings with our supervisors and teaching assistant whenever possible. Team members should come to meetings prepared to give a short update on their progress and discuss any problems they are facing.

If someone knows that they will be late, need to leave early, or cannot attend a meeting, they should let the rest of the team know on Discord as early as possible. Anyone who misses a meeting is responsible for reviewing the meeting notes and catching up on any decisions or tasks that were discussed.

Acceptable Excuse

Acceptable reasons for missing a meeting or deadline include illness, family emergencies, religious observances, or unavoidable academic conflicts such as a test or exam. Planned commitments, such as job interviews or travel, are also understandable as long as the team is informed ahead of time and any affected work is handled beforehand.

Reasons such as forgetting a meeting, oversleeping, or choosing a non-urgent commitment without informing the team would generally not be considered acceptable. Since everyone on the team has other courses and responsibilities, our main expectation is that members communicate early and honestly when something may affect their availability.

In Case of Emergency

If a team member experiences an emergency that prevents them from attending a meeting or completing a task, they should inform the team on Discord as soon as they are reasonably able to. They will not be expected to provide personal details about the situation.

If possible, the member should push any work they have already completed to GitHub and leave a short update explaining what has been done and what is still remaining. This will make it easier for another team member to continue the work if necessary. The team can then redistribute tasks depending on the situation and how close the next deadline is. If the emergency affects a major course deliverable, the team will contact the teaching assistant or instructor to discuss the situation and determine the next steps.

Accountability and Teamwork

Quality

Each team member is expected to make reasonable progress on their assigned tasks and communicate any blockers before they become a larger problem. For written deliverables, we will aim to have individual sections completed at least two days before the final deadline. This will give the team enough time to review the full document, fix inconsistencies, and make sure everything follows the required format.

For development work, code changes will be submitted through pull requests and reviewed by at least one other team member before being merged. Automated checks should also pass before a change is added to the main branch. If someone is unsure about the quality of their work or is having difficulty completing a task, we would rather they raise the issue early so that the team has time to help.

Tasks will be tracked using GitHub issues with an assigned team member and target date so that everyone can see what is currently being worked on. A commitment means an agreed task, meeting, review, or deliverable with a specific due date. A commitment will be considered missed when it is not completed by the agreed date and the team was not notified beforehand, unless an emergency prevented the member from providing notice. Missing two commitments within a rolling four-week period will be considered repeatedly missing commitments. The first step will be to check in with the member privately and understand whether there is a problem or whether they need help. If the issue continues, the team will discuss it together and agree on how to get the work back on track. If the situation still cannot be resolved within the team, we will involve our teaching assistant.

Attitude

We expect everyone on the team to feel comfortable sharing ideas, asking questions, and speaking up when they are unsure about something. Different opinions and approaches are expected, and everyone should treat each other's ideas with respect. When reviewing documents or pull requests, feedback should focus on the work itself and clearly explain what could be improved and why.

Team members are also expected to be reliable and supportive. This means following through on assigned tasks, communicating when something is going wrong, and helping teammates when they need support. We also expect everyone to communicate professionally with our supervisors, teaching assistant, and each other.

Disagreements are a normal part of working as a team. If a disagreement occurs, the people involved should first try to resolve it through a respectful discussion where everyone has a chance to explain their perspective. If the issue affects the project or cannot be resolved directly, the team will discuss it together and follow the decision-making process outlined below. If a conflict continues or becomes personal, we will involve our teaching assistant for additional support.

Stay on Track

To keep the project on schedule, tasks will be tracked through GitHub issues with an assigned team member and target date. This will make it easier for everyone to see what is being worked on, what still needs to be completed, and whether anything is falling behind. During our weekly meetings, we will review our current tasks, discuss any blockers, and adjust responsibilities if someone has too much work.

Larger deliverables will be broken down into smaller tasks with internal deadlines so that problems can be identified before the final submission date. We will also keep track of things such as meeting attendance, completed issues, pull requests, and contributions to deliverables. The purpose of this is not to compare team members, but to make sure the workload remains reasonably balanced throughout the project. As a baseline, each member will aim to contribute approximately 10 hours per week, including scheduled class time. During weekly planning, the team will compare the estimated hours and deadlines for each member's assigned issues. A weekly workload between 8 and 12 hours will generally be considered balanced, with adjustments made for course deadlines, emergencies, and differences in task complexity. If a member's planned workload falls outside this range, the team will redistribute tasks, pair members on larger tasks, or adjust deadlines where possible.

If a team member starts falling behind or is not contributing consistently, another member will first check in with them privately to understand the situation and see if they need help. If the problem continues, the team will discuss

it together and create a clear plan to get back on track, which may include changing responsibilities or setting a new deadline. If the issue still cannot be resolved, we will involve our teaching assistant and the situation may also be reflected in peer evaluations.

We also want to recognize members who put in additional effort or help the team beyond their assigned responsibilities. This can be as simple as acknowledging their contribution during meetings and making sure their work is recognized by the rest of the team.

Team Building

Since all the team members already know each other, we have a good starting point as a team. However, working together on a year-long capstone project will require more communication and coordination than a typical course assignment. We plan to keep communication open and informal through Discord and make sure everyone feels comfortable asking questions or raising concerns.

We will also take time to acknowledge major milestones and completed deliverables, and may occasionally meet outside of project work, such as going for a meal after a major deadline. At the beginning of meetings, members can briefly mention if they have a particularly busy week or other major commitments. This will help us adjust workloads when possible instead of waiting until someone falls behind.

Where possible, we also want team members to work on different areas of the project throughout the year. This will allow everyone to learn more than one part of the system and avoid having one person become responsible for the same type of task for the entire project.

Decision Making

Our team will try to make important decisions through discussion and consensus. When a decision needs to be made, each team member will have the opportunity to share their opinion and explain their reasoning. The goal will be to choose an option that everyone is comfortable moving forward with, even if it is not everyone's first choice.

If we cannot reach an agreement within a reasonable amount of time, we will use a majority vote. Since the team has four members, there is also the possibility of a tie. In this case, we will give more weight to the team member who is working most closely with that part of the system. For larger decisions that could affect the overall direction of the project, we may instead ask our supervisors or teaching assistant for input.

Decisions that depend on CAS requirements, such as where the LLM should be hosted or how certain department data can be used, will be discussed with

our supervisors before being finalized. Important decisions will also be recorded in the meeting notes so that the team can refer back to them later.